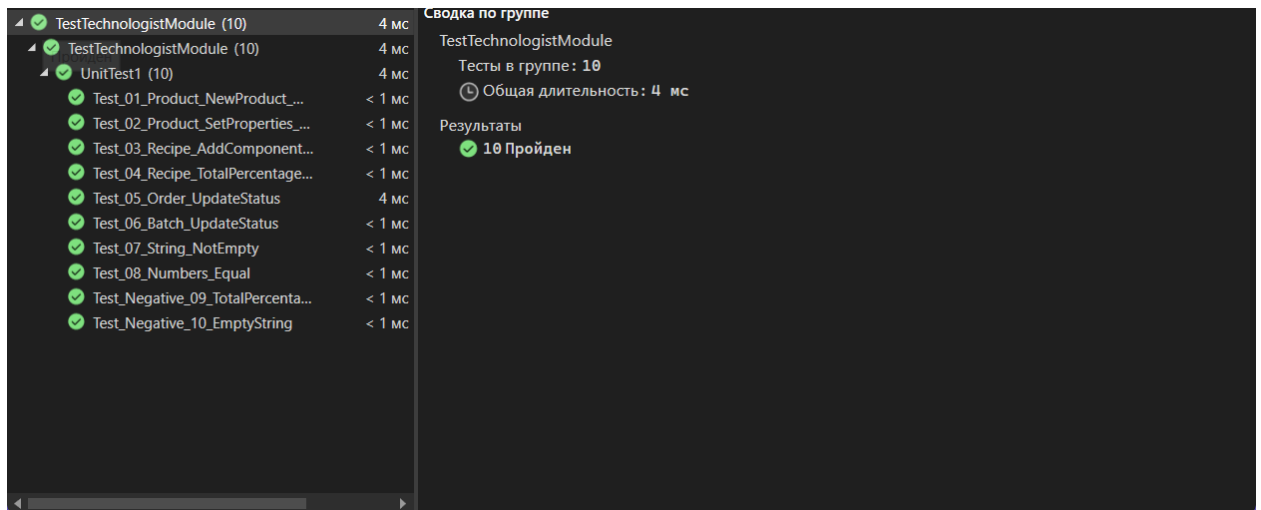




```
using Microsoft.VisualStudio.TestTools.UnitTesting;
```

```
using System.Collections.Generic;
```

```
namespace TestTechnologistModule
```

```
{
```

```
    [TestClass]
```

```
    public class UnitTest1
```

```
    {
```

```
        // ===== ТЕСТОВЫЕ МОДЕЛИ =====
```

```
        public class TestProduct
```

```
        {
```

```
            public long Id { get; set; }
```

```
            public string Code { get; set; }
```

```
            public string Name { get; set; }
```

```
        }
```

```
        public class TestComponent
```

```
        {
```

```
            public string Name { get; set; }
```

```
            public decimal Percentage { get; set; }
```

```
        }
```

```
        // ===== ПОЗИТИВНЫЕ ТЕСТЫ (8 шт.) =====
```

[TestMethod]

public void Test_01_Product_NewProduct_DefaultValues()

```
{  
    var product = new TestProduct();  
  
    Assert.AreEqual(0, product.Id);  
    Assert.IsNull(product.Code);  
    Assert.IsNull(product.Name);  
}
```

[TestMethod]

public void Test_02_Product_SetProperties_ValuesSaved()

```
{  
    var product = new TestProduct { Id = 1, Code = "P001", Name = "Тест" };  
  
    Assert.AreEqual(1, product.Id);  
    Assert.AreEqual("P001", product.Code);  
    Assert.AreEqual("Тест", product.Name);  
}
```

[TestMethod]

public void Test_03_Recipe_AddComponent_CountIncreases()

```
{  
    var components = new List<TestComponent>();  
  
    components.Add(new TestComponent { Name = "Комп1", Percentage = 50 });  
    components.Add(new TestComponent { Name = "Комп2", Percentage = 50 });  
  
    Assert.AreEqual(2, components.Count);  
}
```

[TestMethod]

```
public void Test_04_Recipe_TotalPercentage_SumTo100()
```

```
{
```

```
    var percentages = new List<decimal> { 30, 40, 30 };
```

```
    decimal total = 0;
```

```
    foreach (var p in percentages) total += p;
```

```
    Assert.AreEqual(100m, total);
```

```
}
```

[TestMethod]

```
public void Test_05_Order_UpdateStatus()
```

```
{
```

```
    string status = "planned";
```

```
    status = "in_progress";
```

```
    Assert.AreEqual("in_progress", status);
```

```
}
```

[TestMethod]

```
public void Test_06_Batch_UpdateStatus()
```

```
{
```

```
    string status = "planned";
```

```
    status = "running";
```

```
    Assert.AreEqual("running", status);
```

```
}
```

[TestMethod]

```
public void Test_07_String_NotEmpty()
```

```
{
```

```
    string value = "test";
```

```
    Assert.IsNotNull(value);
```

```

        Assert.IsTrue(value.Length > 0);
    }

[TestMethod]
public void Test_08_Numbers_Equal()
{
    int expected = 100;
    int actual = 100;
    Assert.AreEqual(expected, actual);
}

// ===== НЕГАТИВНЫЕ ТЕСТЫ (2 шт. - 20%) =====

[TestMethod]
public void Test_Negative_09_TotalPercentageNot100()
{
    decimal total = 95m;
    bool isValid = total == 100m;
    Assert.IsFalse(isValid, "95% не равно 100%");
}

[TestMethod]
public void Test_Negative_10_EmptyString()
{
    string value = "";
    bool isValid = !string.IsNullOrEmpty(value);
    Assert.IsFalse(isValid, "Пустая строка невалидна");
}
}
}

```